Project Proposal

Mathys Bitsch Adrian Martinez Gao Romain Labbe

March 27, 2026

1 Introduction

This project is part of the COM-304 Radar Track course, which aims to design and implement a complete radar-based sensing system addressing a real-world problem. In this document, we present our project proposal and outline the main idea we intend to develop. Based on our research, we describe the technical approach we plan to follow for the implementation of the system.

1.1 Problem Statement and Motivation

Modern sensing applications such as presence detection, security monitoring, smart environments, and health tracking typically rely on multiple heterogeneous sensors, including cameras, infrared systems, and IoT devices. While effective, these solutions increase system complexity, cost, and often introduce privacy concerns, particularly when visual data is involved.

Millimeter-wave radar technology offers an attractive alternative by enabling contactless sensing of human presence and motion without capturing any visual information. Unlike camera-based systems, radar only analyzes electromagnetic wave reflections, making it inherently privacy-preserving while still providing rich spatial and dynamic information about the environment.

Our objective is to develop an intelligent radar-based system capable of performing different actions depending on the state of a room. For example, it could adapt ventilation or heating according to room occupancy, turn on lights when a person enters a room, or trigger security alerts at night if the system detects unexpected presence. The system could also generate statistics on room occupancy over time. In addition, we would like to be able

to detect critical situations such as falls, particularly in the case of elderly individuals, and potentially alert emergency services.

1.2 System Overview

The proposed system is a real-time mmWave radar-based sensing platform designed to perceive and interpret human presence and motion in an indoor environment. It takes raw radar measurements as input and progressively transforms them into structured spatial information and high-level semantic outputs.

The system is organized as a modular processing pipeline : First, raw ADC radar data is processed to extract range, velocity, and angle information, which is then converted into a 3D point cloud representation. This representation is further refined through detection and filtering steps, including background removal and CFAR-based target detection. Detected points are then clustered into meaningful objects, which are tracked over time to maintain consistent identities.

Finally, higher-level features are extracted from the tracked trajectories in order to infer occupancy information and human activity patterns. The output of the system consists of continuously updated tracked individuals, along with their trajectories and inferred behaviors, enabling applications such as smart environment control, monitoring, and anomaly detection.

1.3 Existing Implementation and Project Extension

During the preparation of this report, we noticed that a previous group from last year had already developed a similar radar tracking project within the same course. Their implementation is publicly available on GitHub and provided as a reference in the course repository. After discussing with the Radar Tracking teaching assistant, who was part of that previous project, we were informed that it is acceptable to do the same project, provided that meaningful extensions are added.

We therefore considered several possible extensions to the baseline tracking system. A primary objective is to significantly improve tracking robustness by implementing through-wall detection. This involves refining algorithms to detect and track individuals even in scenarios involving obstacles or full occlusion.

Another direction is to estimate the activity level of tracked persons to detect abnormal situations, such as falls in elderly individuals, which could trigger emergency alerts.

We are also considering implementing unfinished features from the previous year's project, specifically body shape reconstruction using machine learning. However, the feasibility of this extension is currently under evaluation. The primary constraint is the lack of a labeled dataset required to effectively train and validate the reconstruction models. Additionally, completing this feature within the allocated timeframe appears to be challenging since we would also need to deeply study the machine leaning techniques involved. This extension appears like a 2-projects in 1 for us.

The detailed implementation of these extensions are presented in the technical sections of this report.

2 Technical implementation

This section describes the setup used in the project, the processing pipeline for the base system (up to tracking), the implementation of the proposed extensions, the metrics used to evaluate system performance, and the main anticipated challenges.

2.1 Hardware and Software setup

The setup uses a TI XWR1843BOOST mmWave radar with a DCA1000EVM acquisition board, connected via USB and Ethernet. The radar is configured and controlled using mmWave Studio, with Lua scripts to define parameters and handle data acquisition.

2.2 Processing Pipeline

As mentioned in section 1.2, we aim to design our system software as a large modular pipeline. The details of each component of this pipeline are described below.

2.2.1 Radar hardware processing

The radar hardware already performs the mixing and low-pass filtering of the signal. This analog processing extracts the beat frequency before digitization,

providing the raw data used for our software-based processing pipeline.

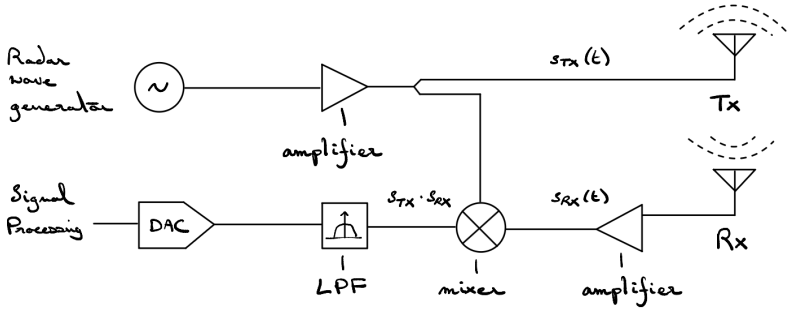

Figure 1: Radar Hardware Processing Chain

2.2.2 Point Cloud Construction

The pipeline begins with the acquisition of raw ADC data. A Range FFT is first applied to estimate distance, followed by a Doppler FFT to extract relative velocities. Angle estimation is then performed using beamforming (azimuth and elevation) to complete the spatial localization.

At this stage, each detection is initially described in a spherical range-velocity-angle space (r, v, θ, ϕ). These coordinates are then converted into a final Cartesian point cloud matrix defined by (x, y, z, v, P). This multidimensional structure serves as the primary input for all subsequent filtering and detection algorithms.

2.2.3 Static Clutter Removal

A background removal stage is applied to the Range-Doppler matrix to eliminate static reflections. This can be achieved either by suppressing zero-Doppler components, which correspond to stationary objects, or by estimating and subtracting a temporal average background.

The latter approach relies on the assumption that static clutter remains consistent over time, allowing it to be isolated and removed from the signal. These methods effectively reduce interference from stationary obstacles (e.g., walls and furniture), enabling the system to focus on moving targets while also reducing computational load.

2.2.4 Constant False Alarm Rate (CFAR)

To isolate relevant detections from noise, a CFAR algorithm is applied. It adaptively calculates a local threshold based on the surrounding noise level to validate only the peaks with a significant signal power (P). This step further cleans the point cloud before spatial grouping.

2.2.5 Clustering (DBSCAN)

The remaining points are grouped into distinct entities using the DBSCAN algorithm. It identifies clusters based on spatial proximity (x,y,z) and velocity similarity (v). Points that do not belong to a dense cluster are discarded as outliers, effectively isolating individual targets.

2.2.6 Multi-Target Tracking (GTrack)

The clustered points are processed by the GTrack algorithm to ensure temporal consistency across frames. This stage relies on a Kalman Filter to predict the future position and velocity of each cluster based on its previous state.

The algorithm performs two main functions:

- Data Association: It links new clusters to existing tracks by comparing their predicted states with the current measurements.
- Track Management: It handles the creation of new tracks for newly appearing objects, as well as the removal of tracks for objects that leave the radar's field of view.

2.2.7 Visualization and System Output

The final stage involves the real-time projection of processed data onto a graphical interface. The system generates a dynamic 3D scatter plot, where points are color-coded based on velocity or signal intensity.

Two main outputs are provided:

- Point Cloud View: Displays both raw and filtered point clouds, allowing visualization of the environment's geometric structure.
- Centroid and Track Visualization: Overlays centroids or bounding boxes on detected clusters, illustrating their trajectories over time along with the associated activity labels (e.g., Walking).

This visualization layer serves both as a debugging tool for validating the accuracy of the processing pipeline and as the final user interface for monitoring human activities.

2.2.8 Hardware-Based Feedback (Extra)

Finally, we aim to integrate simple hardware-based outputs to provide a more concrete representation of the system, rather than relying solely on on-screen data.

This feature is not expected to be technically complex. For example, an Arduino board can be connected to the data-processing computer, enabling serial communication between the system and the hardware components. This setup would allow basic interactions, such as turning an LED on or off depending on room occupancy. Additionally, a fan could be included, with its speed adjusted according to temperature measurements obtained from a simple DHT11 sensor, as well as the number of occupants. All required hardware components will be provided.

These elements are primarily intended as demonstrative enhancements rather than essential functionalities. Therefore, they should be treated as optional features and implemented only if time allows.

2.3 Technical implementation of extensions

We propose three possible extensions for the project:

2.3.1 Extension 1 : Analysis for Fall Detection and Emergency Response

The system implements a real-time monitoring layer to detect falls. By buffering 3D coordinates, we calculate instantaneous velocity and acceleration to identify distress patterns. A fall is defined by a rapid vertical transition:

$$V_z = \frac{\Delta Z}{\Delta t} \tag{1}$$

Detection triggers when a fast downward movement (high V_z) is followed immediately by no movement at ground level ($Z \approx 0$).

The interface also includes a visual trail that shows the person's previous movements, fading over time, so it is easy to see what happened before the

fall. This rule-based approach remains simple and interpretable but a more robust and perhaps reliable alternative would be to use machine learning models to automatically recognize fall patterns. However, this would require collecting a sufficiently large and diverse dataset of normal activities and fall events.

2.3.2 Extension 2 : Through-Wall Detection

We would like to introduce a physical obstacle (a wall) between the sensors and the targets. The challenge here is to manage signal attenuation and multi-path reflections caused by the wall. We aim to refine our processing algorithms to detect occupants behind the wall, validating the system's ability to maintain an accurate count and position estimate despite the obstacles. The power received (P_r) from a person behind a wall is affected by the "Two-Way Transmission Coefficient" (L_{wall}):

$$P_r = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4 \cdot L_{wall}^2}$$

P_t**:** Transmitted power.

G**:** Antenna gain.

λ**:** Wavelength $c/f \approx 0.00389\,\text{m}$ (3.89 mm).

σ**:** Radar Cross Section (how "reflective" a human is); ranges from 0.5 to 1.0 m.

L_{wall}**:** The loss factor caused by the wall material (brick, concrete, etc.). For a wall of 15 cm concrete, there is approximately a 15 dB loss, becoming 30 dB due to the back-and-forth signal. This leads to a ratio around 1000 (making the signal 1000 times weaker). Using a material like plywood results in a much lower loss factor, with a signal only 2 to 4 times weaker $\Rightarrow L^2 \approx 4$.

R**:** The range (distance) collected from the radar data.

2.3.3 Extension 3: Body Shape Reconstruction (Previous Year Project)

This extension builds upon the objectives of the previous project group, with a focus on reconstructing a simplified human body structure (e.g., head, torso, and limbs). The project proposal from last year can be found in the resources section of this document.

The implementation would involve training a neural network using existing datasets that may be relevant to our use case, such as the MPII Human Pose Dataset, which provides annotated human pose data.

However, we remain cautious regarding the feasibility of this extension due to significant technical challenges, particularly the lack of directly applicable radar-specific training data and the associated dataset limitations. These challenges are further detailed in the *Anticipated Challenges* section below.

2.4 Evaluation Methods

To evaluate and fine-tune the system, we define here some metrics that could guide the development and optimization of the pipeline.

- **Signal-to-Noise Ratio (SNR):** Measures the quality of the received signal compared to background noise:

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}$$

  A higher SNR indicates better target visibility after clutter removal.

- **Detection Count Stability:** Evaluates the temporal consistency of the system by monitoring the number of detected points per frame. Large fluctuations may indicate unstable filtering or clustering.

- **Track Continuity (Track Fragmentation):** Measures how often a tracked object is interrupted over time. A lower fragmentation rate indicates more stable tracking.

- **Computational Performance:** Measured using processing latency per frame and frame rate (FPS). The system is designed to operate in real time, meaning FPS must remain sufficiently high. A drop in FPS

typically indicates inefficient or non-optimized parts of the processing pipeline.

These metrics allow us to evaluate the trade-off between accuracy and real-time performance.

2.5 Anticipated Challenges

Several challenges are expected in the execution of this project. Radar data is inherently noisy and sensitive to environmental clutter, which can impact detection reliability and spatial resolution when distinguishing multiple targets. We may consider using more than one radar to improve robustness and reliability, but this would come at the cost of increased system complexity.

Specific challenges arise within the presented extensions:

- **Fall Detection Analysis:** Extracting useful motion features from sparse point cloud data is difficult. It is also hard to tell the difference between a fast normal movement (like sitting down) and a real fall, so the velocity thresholds must be carefully adjusted.

- **Through-Wall Detection:** Operating in Non-Line-of-Sight (NLOS) conditions introduces significant challenges due to strong signal attenuation caused by wall materials such as concrete, brick, or reinforced structures. These materials induce high path loss through absorption, scattering, and reflection, which can severely reduce the received signal power. As a result, maintaining a sufficient Signal-to-Noise Ratio (SNR) becomes critical for reliable detection. In addition to attenuation, multipath propagation is significantly more complex as signals undergo multiple reflections and diffractions before reaching the receiver.

- **Body Shape Reconstruction** Lack of data. A lot of effort for the given time (2-in-1 project). Technical differences between our specific radar hardware and the configurations used in these external datasets that may lead to poor accuracy in our environment. Additionally, the high computational power required for real-time 3D shape reconstruction might exceed our system's limits.

3 Expected Project Timeline

Our goal would be to follow this timeline approximetely. The project progress mentioned for each week is supposed to be finished on Tuesday of the corresponding week.

Week	**Date**	**Project Progress / Tasks**	**Comments**
Week 7	Tue. 31 Mar.	Feedback on Project Proposal	
Week 8	Tue. 7 Apr.	Hardware setup and starting implementation	Break
Week 9	Tue. 14 Apr.	Hardware setup, obtaining real time results	
Week 10	Tue. 21 Apr.	Data Reconstruction (section 2.2.2)	Progress report due
Week 11	Tue. 28 Apr.	Filtering and Clustering (2.2.3, 2.2.4, 2.2.5)	
Week 12	Tue. 5 May	Real Time Tracking (2.2.6)	
Week 13	Tue. 12 May	Implementing chosen extension	
Week 14	Tue. 19 May	Implementing chosen extension	
Week 15	Tue. 26 May	General review (doc, prep)	Final presentation, report and demo

Table 1: Project timeline

4 Resources

The resources below will be potentially used in our project:

- Grouped People Counting Using mm-Wave FMCW MIMO Radar
- People Counting in a COVID-19 and GDPR Context using IR-UWB radar signals (GitHub Repo)
- Real-time mmWave Multi-Person Pose Estimation System for Privacy-Aware Windows (GitHub Repo)

- OpenMMLab Pose Estimation Toolbox and Benchmark (GitHub Repo)
- Household Radar Can See Through Walls and Knows How You're Feeling
- Real-Time Human Tracking with mmWave Radar (a past EPFL radar project)
- A Multitask Network for People Counting, Motion Recognition, and Localization Using Through-Wall Radar
- Computer Vision and Machine Learning from MPI